

DATA STRUCTURES

Complete Handwritten-Style Exam Notes

- Unit I – Arrays, Searching & Sorting
- Unit II – Linked Lists & Applications
- Unit III – Stacks, Queues, Recursion
 - Unit IV – Trees & Graphs

Prepared by

MANTHAN SAHU

Rajendra University, Balangir | B.Sc. Computer Science | Semester II

Reference Text: *Classic Data Structures* — D. Samanta, PHI, 2/e

"Notes designed to be easy to understand and exam-ready — with definitions, diagrams, dry-run examples, code and quick-revision boxes."

TABLE OF CONTENTS

UNIT I — Arrays, Searching & Sorting

- 1.1 Introduction to Data Structures & Classification
- 1.2 One-Dimensional Arrays & Memory Representation
- 1.3 Array Operations (Traverse, Search, Insert, Delete, Merge)
- 1.4 Two-Dimensional & Multidimensional Arrays
- 1.5 Linear Search vs Binary Search
- 1.6 Sorting — Selection Sort & Bubble Sort

UNIT II — Linked Lists

- 2.1 Singly Linked List — Definition & Memory Representation
- 2.2 Operations: Traverse, Search, Insert, Delete, Merge
- 2.3 Doubly Linked List & its Operations
- 2.4 Circular & Doubly Circular Linked List
- 2.5 Applications: Sparse Matrix & Polynomial Representation

UNIT III — Stacks, Queues & Recursion

- 3.1 Stack — Array & Linked Representation
- 3.2 Infix–Postfix Conversion & Postfix Evaluation
- 3.3 Recursion (Factorial, Fibonacci) & Tower of Hanoi
- 3.4 Queue — Array & Linked Representation
- 3.5 Circular Queue, Deque & Priority Queue

UNIT IV — Trees & Graphs

- 4.1 Tree Terminology & Binary Tree Properties
- 4.2 Tree Traversals (Inorder, Preorder, Postorder)
- 4.3 Binary Search Tree, AVL Tree, M-Way Search Tree
- 4.4 Graph Terminology & Representations
- 4.5 BFS & DFS Traversal, Applications

QUICK REVISION

- Complexity Cheat-Sheet (Time & Space)
- Important Exam Questions — Unit-wise

UNIT I

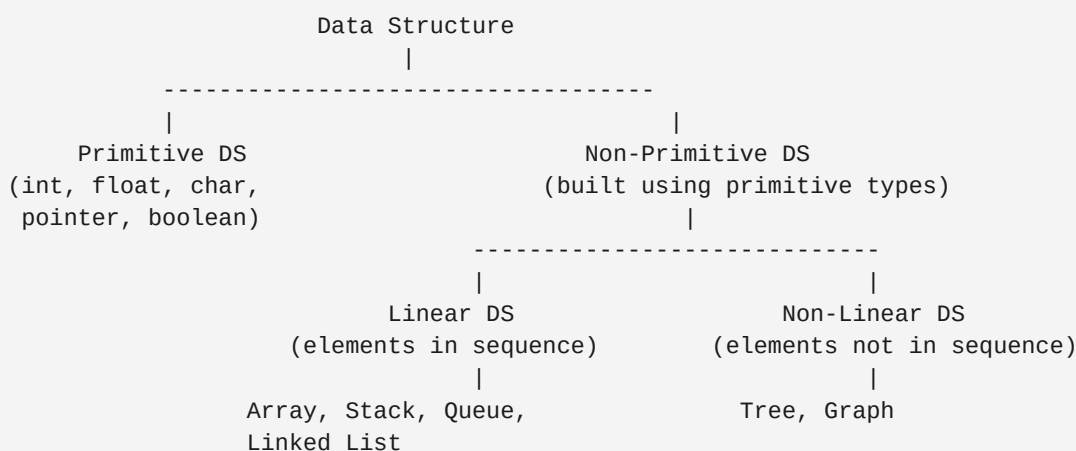
Arrays, Searching & Sorting

1.1 Introduction to Data Structures

Definition: A **Data Structure** is a systematic way of organizing, storing and managing data in a computer so that it can be used efficiently. It defines the relationship between data elements and the operations that can be performed on them.

Classification of Data Structures

Fig 1.1 — Classification of Data Structures



- **Linear Data Structure:** Elements are arranged in a sequential/linear order, and each element (except first/last) has a unique predecessor and successor. Example: Array, Stack, Queue, Linked List.
- **Non-Linear Data Structure:** Elements are not arranged sequentially; every element may connect to more than one other element. Example: Tree, Graph.
- **Static DS:** Size is fixed at compile time (e.g. Array).
- **Dynamic DS:** Size can grow/shrink at run time (e.g. Linked List).

★ **Exam Tip:** A very common 2-mark question is “Differentiate between Linear and Non-Linear Data Structures” or “Classify Data Structures with a diagram.” Always draw the tree diagram above — it fetches easy marks.

1.2 One-Dimensional Array & Memory Representation

Definition: An **Array** is a collection of homogeneous (same type) data elements stored in contiguous (continuous) memory locations, referred by a common name and accessed using an index/subscript.

Memory Representation

If an array A is declared as $A[\text{lower_bound} \dots \text{upper_bound}]$, and each element takes 'w' bytes, and the base address (address of first element) is B, then the address of the i-th element is calculated as:

Formula — 1-D Array Address

$$\text{Address}(A[i]) = B + w * (i - \text{lower_bound})$$

Example: int A[5] with Base Address B = 1000, w = 2 bytes (size of int)
 $\text{Address}(A[3]) = 1000 + 2 * (3 - 0) = 1006$

Example — Address Calculation

An array A[10] of integers (2 bytes each) is stored with base address 2000. Find the address of A[6].
 $\text{Address}(A[6]) = 2000 + 2 * (6 - 0) = \mathbf{2012}$

Array Operations

(a) Traversing: Visiting each element of the array exactly once to process it.

Program / Pseudocode

```
void traverse(int A[], int n) {
    for (int i = 0; i < n; i++)
        printf("%d ", A[i]);
}
```

(b) Searching: Finding the location of a given element (see Section 1.5 for Linear Search & Binary Search in detail).

(c) Insertion: Adding a new element at a given position. All elements from that position onward must be shifted one place to the right.

Program / Pseudocode

```
void insert(int A[], int *n, int pos, int item) {
    int i;
    for (i = *n - 1; i >= pos; i--)
        A[i + 1] = A[i];      // shift elements right
    A[pos] = item;           // place new item
    (*n)++;
}
```

Fig 1.2 — Insertion in Array

Before insert(A, pos=2, item=25):

Index :	0	1	2	3	4
A :	[10]	[20]	[30]	[40]	[]

Shift A[2..3] right by one place --->

Index :	0	1	2	3	4
A :	[10]	[20]	[]	[30]	[40]

Insert 25 at index 2 --->

Index :	0	1	2	3	4
A :	[10]	[20]	[25]	[30]	[40]

(d) Deletion: Removing an element from a given position. All elements after that position are shifted one place to the left.

Program / Pseudocode

```

int delete(int A[], int *n, int pos) {
    int item = A[pos];
    for (int i = pos; i < *n - 1; i++)
        A[i] = A[i + 1];    // shift elements left
    (*n)--;
    return item;
}

```

(e) Merging: Combining two sorted arrays into a single sorted array.

Program / Pseudocode

```

void merge(int A[], int n, int B[], int m, int C[]) {
    int i = 0, j = 0, k = 0;
    while (i < n && j < m) {
        if (A[i] <= B[j]) C[k++] = A[i++];
        else C[k++] = B[j++];
    }
    while (i < n) C[k++] = A[i++];
    while (j < m) C[k++] = B[j++];
}

```

Example — Merging

A = [2, 5, 9] and B = [1, 4, 6, 10] → Merged C = [1, 2, 4, 5, 6, 9, 10]

Operation	Best Case	Worst Case
Traverse	O(n)	O(n)
Insert (at end)	O(1)	O(1)
Insert (at beginning/middle)	O(1)	O(n)
Delete (at end)	O(1)	O(1)
Delete (at beginning/middle)	O(1)	O(n)
Merge (two sorted arrays)	O(n+m)	O(n+m)

1.3 Two-Dimensional & Multidimensional Arrays

Definition: A **Two-Dimensional Array** is an array of arrays — data is organised in rows and columns, declared as A[m][n] where m = number of rows, n = number of columns.

Memory Representation of 2-D Array

Since main memory is linear (1-D), a 2-D array is stored using either **Row-Major Order** (row by row) or **Column-Major Order** (column by column).

Formula — 2-D Array Address

Row-Major Order (used by C, C++, Java):

$$\text{Address}(A[i][j]) = B + w * [(i - \text{lb}_r) * N + (j - \text{lb}_c)]$$

where N = number of columns

Column-Major Order (used by FORTRAN):

$$\text{Address}(A[i][j]) = B + w * [(j - \text{lb}_c) * M + (i - \text{lb}_r)]$$

where M = number of rows

Example — Row-Major Address Calculation

$A[4][5]$ (0-indexed) stored in row-major order, base address $B = 1000$, $w = 4$ bytes. Find address of $A[2][3]$.

Address = $1000 + 4 * [(2-0)*5 + (3-0)] = 1000 + 4*(10+3) = 1000 + 52 = \mathbf{1052}$

Multidimensional Array: Generalisation of 2-D arrays to n dimensions (e.g. $A[l_1][l_2][l_3] \dots [l_n]$), commonly used for 3-D matrices, tensors, etc. Address calculation extends the same row-major principle across all dimensions.

★ **Exam Tip:** Row-major vs column-major address formula numericals are asked almost every semester (5–10 marks). Memorise the formula and practice at least 2–3 numericals.

1.4 Linear Search vs Binary Search

Linear Search

Definition: Linear Search checks every element of the array one by one, from the beginning, until the desired element is found or the array ends. Works on both sorted and unsorted arrays.

Program / Pseudocode

```
int linearSearch(int A[], int n, int key) {
    for (int i = 0; i < n; i++)
        if (A[i] == key)
            return i;           // found at index i
    return -1;                  // not found
}
```

Binary Search

Definition: Binary Search repeatedly divides a **sorted** array into halves, comparing the key with the middle element, and searching only the relevant half — much faster than linear search.

Program / Pseudocode

```

int binarySearch(int A[], int n, int key) {
    int low = 0, high = n - 1;
    while (low <= high) {
        int mid = (low + high) / 2;
        if (A[mid] == key) return mid;
        else if (A[mid] < key) low = mid + 1;
        else high = mid - 1;
    }
    return -1;
}

```

Example — Binary Search Dry Run

Sorted array: [2, 5, 8, 12, 16, 23, 38, 45, 56, 72] Key = 23

Step 1: low=0, high=9, mid=4 → A[4]=16 < 23 → low=5

Step 2: low=5, high=9, mid=7 → A[7]=45 > 23 → high=6

Step 3: low=5, high=6, mid=5 → A[5]=23 = key → **Found at index 5**

Parameter	Linear Search	Binary Search
Pre-condition	Array may be unsorted	Array must be sorted
Approach	Sequential	Divide and Conquer
Time Complexity (Best)	O(1)	O(1)
Time Complexity (Worst)	O(n)	O(log n)
Space Complexity	O(1)	O(1)
Suitable for	Small / unsorted data	Large sorted data

1.5 Sorting — Selection Sort & Bubble Sort

Selection Sort

Definition: Selection Sort repeatedly selects the **minimum** element from the unsorted part of the array and places it at the beginning (swaps it into position).

Program / Pseudocode

```

void selectionSort(int A[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int min_idx = i;
        for (int j = i + 1; j < n; j++)
            if (A[j] < A[min_idx])
                min_idx = j;
        // swap A[i] and A[min_idx]
        int t = A[i]; A[i] = A[min_idx]; A[min_idx] = t;
    }
}

```

Example — Selection Sort Dry Run

Array: [29, 10, 14, 37, 13]
 Pass 1: min=10 → [10, 29, 14, 37, 13]
 Pass 2: min=13 → [10, 13, 14, 37, 29]
 Pass 3: min=14 (already in place) → [10, 13, 14, 37, 29]
 Pass 4: min=29 → **[10, 13, 14, 29, 37]** (Sorted)

Bubble Sort

Definition: Bubble Sort repeatedly compares **adjacent** elements and swaps them if they are in the wrong order — smaller/larger elements 'bubble up' to their correct position with each pass.

Program / Pseudocode

```
void bubbleSort(int A[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int swapped = 0;
        for (int j = 0; j < n - i - 1; j++) {
            if (A[j] > A[j + 1]) {
                int t = A[j]; A[j] = A[j + 1]; A[j + 1] = t;
                swapped = 1;
            }
        }
        if (!swapped) break; // already sorted - optimisation
    }
}
```

Example — Bubble Sort Dry Run

Array: [5, 1, 4, 2, 8]
 Pass 1: [1, 4, 2, 5, 8] (compare & swap adjacent pairs)
 Pass 2: [1, 2, 4, 5, 8]
 Pass 3: [1, 2, 4, 5, 8] — no swaps, sorting complete → **Sorted**

Algorithm	Best Case	Average Case	Worst Case	Space	Stable?
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes

★ **Exam Tip:** Remember: Selection sort minimises the number of SWAPS (good when swap cost is high); Bubble sort with the 'swapped' flag gives $O(n)$ best case on an already-sorted array. Dry-run questions on 5–6 element arrays are very common.

Full Pass-by-Pass Trace with Comparison Count (Selection Sort)

Array: [29, 10, 14, 37, 13] ($n = 5$). Selection sort always performs $n(n-1)/2$ comparisons regardless of input order, but only up to $(n-1)$ swaps.

Pass	Comparisons	Swap made	Array after pass
1	4 ($i=0$ vs 1,2,3,4)	$29 \leftrightarrow 10$	[10, 29, 14, 37, 13]
2	3 ($i=1$ vs 2,3,4)	$29 \leftrightarrow 13$	[10, 13, 14, 37, 29]

Pass	Comparisons	Swap made	Array after pass
3	2 (i=2 vs 3,4)	none needed	[10, 13, 14, 37, 29]
4	1 (i=3 vs 4)	37 ↔ 29	[10, 13, 14, 29, 37]

Full Pass-by-Pass Trace (Bubble Sort)

Array: [5, 1, 4, 2, 8] (n = 5)

Pass	Comparisons made	Array after pass
1	(5,1)swap (5,4)swap (5,2)swap (5,8)no	[1, 4, 2, 5, 8]
2	(1,4)no (4,2)swap (4,5)no	[1, 2, 4, 5, 8]
3	(1,2)no (2,4)no — no swaps → STOP (optimised)	[1, 2, 4, 5, 8]

1.6 Additional Concepts on Arrays & Data Structures

Advantages & Disadvantages of Array

Advantages	Disadvantages
Random/direct access using index — $O(1)$	Size is fixed at declaration (static)
Simple and easy to implement	Insertion/Deletion is costly (needs shifting)
Memory is used efficiently (no extra pointer)	Wastage of memory if array is not fully used
Good cache locality (contiguous memory)	Cannot easily grow beyond declared size

Basic Operations Performed on Any Data Structure

Operation	Meaning
Create	Declare and allocate memory for the data structure
Insertion	Adding a new data element
Deletion	Removing an existing data element
Traversal	Accessing/visiting each element exactly once
Searching	Finding the location of a given element
Sorting	Arranging elements in ascending/descending order
Merging	Combining two data structures into one

2-D Array — Column-Major Order Worked Example

Example — Column-Major Address Calculation

A[4][5] (0-indexed) stored in **column-major** order, base address B = 1000, w = 4 bytes. Find address of A[2][3]. (M = number of rows = 4)

Address = $B + w * [(j - lb_c) * M + (i - lb_r)] = 1000 + 4 * [(3-0)*4 + (2-0)] = 1000 + 4 * (12+2) = 1000 + 56 = 1056$

Compare with row-major result of A[2][3] = 1052 (Section 1.3) — the two orders give different addresses for the same logical element.

Three-Dimensional Array — Address Formula

3-D Array Address Formula

For A[l1][l2][l3] with dimensions stored in row-major order:

Address(A[i][j][k]) = $B + w * [i * (N2 * N3) + j * N3 + k]$

where N2 = number of columns (2nd dimension size),
N3 = number of depth elements (3rd dimension size)

Complexity Derivation — Binary Search

Binary Search halves the search space at every comparison. If T(n) is the number of comparisons needed for n elements:

Recurrence Relation for Binary Search

$$T(n) = T(n/2) + 1$$

$$T(n/2) = T(n/4) + 1 \Rightarrow T(n) = T(n/4) + 2$$

In general, after k halvings: $T(n) = T(n / 2^k) + k$

The recursion ends when $n / 2^k = 1 \Rightarrow k = \log_2(n)$

Therefore: $T(n) = O(\log n)$

★ **Exam Tip:** The recurrence-relation derivation of Binary Search's $O(\log n)$ complexity is a popular 'derive/prove' theory question worth 5 marks — memorise the steps above.

Unit I — Important Exam Questions

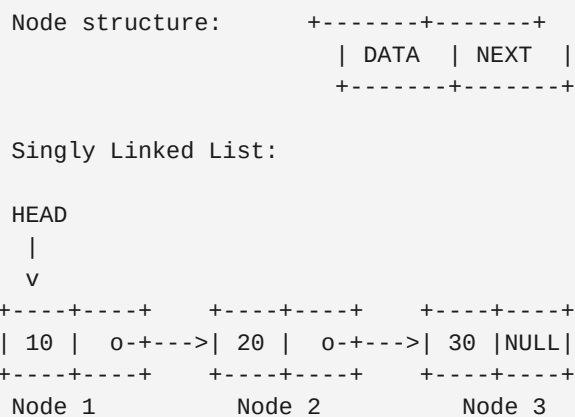
- 1 Define data structure. Explain its classification with a diagram.
- 2 What is an array? Derive the address of an element in a 1-D array with an example.
- 3 Derive the formula for address calculation of an element in a 2-D array using row-major order. Solve a numerical.
- 4 Write an algorithm/program to insert and delete an element in an array.
- 5 Differentiate between linear search and binary search.
- 6 Write and explain the binary search algorithm with a suitable example.
- 7 Sort the array [63, 42, 27, 15, 8] using selection sort and bubble sort. Show all passes.
- 8 Compare selection sort and bubble sort on the basis of time complexity.

UNIT II Linked Lists

2.1 Singly Linked List — Definition & Memory Representation

Definition: A **Linked List** is a linear data structure in which elements (called **nodes**) are not stored in contiguous memory locations. Each node contains two parts — **data** and a **pointer (link)** to the next node. The list is accessed through a pointer called **HEAD/START**, and the last node points to **NULL**.

Fig 2.1 — Singly Linked List



Node Declaration in C

```

struct Node {
    int data;
    struct Node *next;
};
struct Node *head = NULL;

```

Array vs Linked List

Basis	Array	Linked List
Memory	Contiguous	Non-contiguous (scattered)
Size	Fixed (static)	Dynamic (grows/shrinks at run time)
Access	Direct/Random access $O(1)$	Sequential access only $O(n)$
Insertion/Deletion	Costly (shifting needed)	Easy (only pointers changed)
Memory Overhead	None	Extra memory for pointer/link field

2.2 Operations on Singly Linked List

(a) Traversing

Program / Pseudocode

```
void traverse(struct Node *head) {
    struct Node *p = head;
    while (p != NULL) {
        printf("%d -> ", p->data);
        p = p->next;
    }
    printf("NULL\n");
}
```

(b) Searching

Program / Pseudocode

```
struct Node* search(struct Node *head, int key) {
    struct Node *p = head;
    while (p != NULL) {
        if (p->data == key) return p;    // found
        p = p->next;
    }
    return NULL;                        // not found
}
```

(c) Insertion

Insertion can be done at the **beginning**, **end**, or at a **given position**.

Program / Pseudocode

```
// Insert at beginning
void insertBeg(struct Node **head, int val) {
    struct Node *n = malloc(sizeof(struct Node));
    n->data = val;
    n->next = *head;
    *head = n;
}

// Insert at end
void insertEnd(struct Node **head, int val) {
    struct Node *n = malloc(sizeof(struct Node));
    n->data = val; n->next = NULL;
    if (*head == NULL) { *head = n; return; }
    struct Node *p = *head;
    while (p->next != NULL) p = p->next;
    p->next = n;
}
```

Fig 2.2 — Insertion at Beginning

Insert 15 at beginning:

Before: HEAD -> [20] -> [30] -> NULL

After : HEAD -> [15] -> [20] -> [30] -> NULL

(d) Deletion

Program / Pseudocode

```
// Delete first node
void deleteBeg(struct Node **head) {
    if (*head == NULL) return;
    struct Node *temp = *head;
    *head = (*head)->next;
    free(temp);
}
```

(e) Merging Two Linked Lists

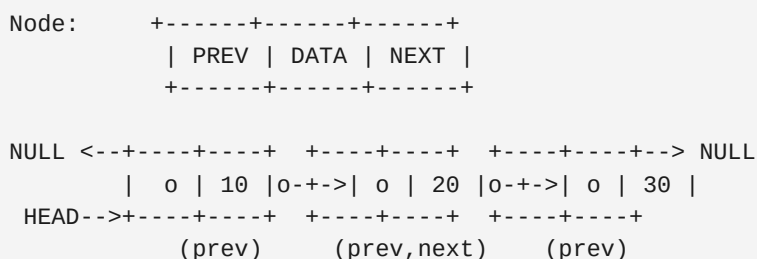
Merging appends/combines two linked lists into a single list — either by attaching the second list at the end of the first (simple concatenation) or by merging two already-sorted lists in sorted order (similar logic to merging sorted arrays).

★ **Exam Tip:** For 'write a program' questions, always show the **struct Node** declaration first, then the function, and finally draw a before/after diagram — examiners give separate marks for the diagram.

2.3 Doubly Linked List (DLL)

Definition: A **Doubly Linked List** is a linked list in which each node has **three** fields: a pointer to the **previous** node, the **data**, and a pointer to the **next** node. This allows traversal in both forward and backward directions.

Fig 2.3 — Doubly Linked List



Program / Pseudocode

```
struct DNode {
    struct DNode *prev;
    int data;
    struct DNode *next;
};
```

Insertion in DLL (at beginning)

Program / Pseudocode

```
void insertBeg(struct DNode **head, int val) {
    struct DNode *n = malloc(sizeof(struct DNode));
    n->data = val;
    n->prev = NULL;
    n->next = *head;
    if (*head != NULL) (*head)->prev = n;
    *head = n;
}
```

Deletion in DLL

Program / Pseudocode

```
void deleteNode(struct DNode **head, struct DNode *del) {
    if (*head == del) *head = del->next;
    if (del->next != NULL) del->next->prev = del->prev;
    if (del->prev != NULL) del->prev->next = del->next;
    free(del);
}
```

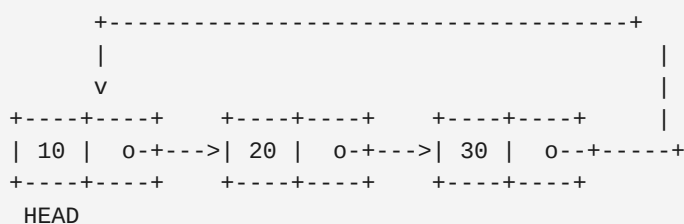
★ **Exam Tip:** DLL advantage: can be traversed in both directions and deletion is more efficient (no need to keep a separate 'previous' pointer while traversing). Disadvantage: extra memory needed for the prev pointer in every node.

2.4 Circular & Doubly Circular Linked List

Circular Linked List (CLL)

Definition: In a **Circular Linked List**, the last node's next pointer points back to the **first node** instead of NULL — forming a circle. It can be traversed continuously without reaching a NULL.

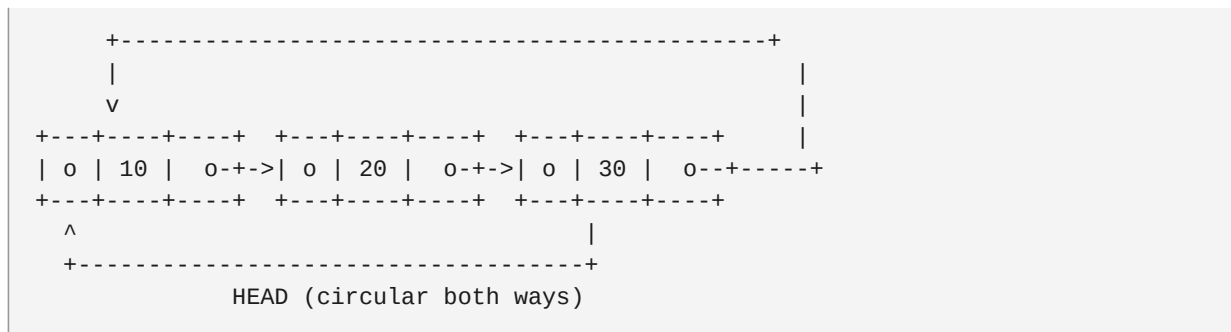
Fig 2.4 — Circular Linked List



Doubly Circular Linked List (DCLL)

Definition: A **Doubly Circular Linked List** combines both concepts: every node has prev and next pointers, AND the last node's next points to the first node while the first node's prev points to the last node.

Fig 2.5 — Doubly Circular Linked List



Key Operations

- **Traversing:** Start from HEAD, move using 'next' until we come back to HEAD (use a do-while loop, not while, since there is no NULL).
- **Insertion:** New node's next/prev are adjusted, and the circular link (last $\leftrightarrow$ first) is maintained after insertion.

List Type	Traversal	Extra Memory	Use Case
Singly Linked List	Forward only	1 pointer/node	Simple sequential data
Doubly Linked List	Forward & Backward	2 pointers/node	Browser history, undo-redo
Circular Linked List	Continuous loop	1 pointer/node	Round-robin CPU scheduling
Doubly Circular LL	Continuous, both ways	2 pointers/node	Music playlist (loop)

2.5 Applications of Linked List

(a) Sparse Matrix Representation

Definition: A **Sparse Matrix** is a matrix in which most of the elements are zero. Storing it as a normal 2-D array wastes memory, so only the **non-zero elements** are stored using a linked list, each node storing (row, column, value).

Fig 2.6 — Sparse Matrix using Linked List

Sparse Matrix (4x4):		Linked List representation (stores only non-zero elements)
0 0 3 0	-->	Row Col Val
0 0 0 0		0 2 3 -> 2 1 8 -> 3 0 6 -> NULL
0 8 0 0		
6 0 0 0		

Program / Pseudocode

```
struct SparseNode {
    int row, col, value;
    struct SparseNode *next;
};
```

(b) Polynomial Representation

Definition: A polynomial such as $4x^3 + 6x^2 + 5$ can be represented as a linked list where each node stores the **coefficient** and **exponent** of one term, linked in decreasing order of exponent.

Fig 2.7 — Polynomial as Linked List

Polynomial: $4x^3 + 6x^2 + 5$ (i.e. $4x^3 + 6x^2 + 0x^1 + 5x^0$)

```

+-----+-----+   +-----+-----+   +-----+-----+
|coef|exp|   |coef|exp|   |coef|exp|
|  4  | 3  |o-->|  6  | 2  |o-->|  5  | 0  |o--> NULL
+-----+-----+   +-----+-----+   +-----+-----+

```

Program / Pseudocode

```

struct PolyNode {
    int coeff, expo;
    struct PolyNode *next;
};

```

Example — Polynomial Addition

$P1 = 5x^2 + 3x + 2$ $P2 = 4x^2 + 7$

Traverse both lists; if exponents match, add coefficients, else copy the term with the higher exponent.

Result: $9x^2 + 3x + 9$

★ **Exam Tip:** Sparse matrix and polynomial representation are classic 5-mark 'application' questions — always mention WHY linked list saves memory compared to a 2-D array or simple coefficient array.

2.6 Additional Concepts on Linked Lists

Insertion at a Given Position & End (Singly Linked List)

Program / Pseudocode

```
// Insert at a given position 'pos' (0-indexed)
void insertPos(struct Node **head, int pos, int val) {
    struct Node *n = malloc(sizeof(struct Node));
    n->data = val;
    if (pos == 0) { n->next = *head; *head = n; return; }
    struct Node *p = *head;
    for (int i = 0; i < pos - 1 && p != NULL; i++)
        p = p->next;
    n->next = p->next;
    p->next = n;
}

// Delete a node at a given position
void deletePos(struct Node **head, int pos) {
    if (*head == NULL) return;
    struct Node *temp = *head;
    if (pos == 0) { *head = temp->next; free(temp); return; }
    struct Node *prev = NULL;
    for (int i = 0; i < pos && temp != NULL; i++) {
        prev = temp; temp = temp->next;
    }
    prev->next = temp->next;
    free(temp);
}
```

Doubly Linked List — Insertion at End & Full Traversal (Both Directions)

Program / Pseudocode

```
void insertEnd(struct DNode **head, int val) {
    struct DNode *n = malloc(sizeof(struct DNode));
    n->data = val; n->next = NULL;
    if (*head == NULL) { n->prev = NULL; *head = n; return; }
    struct DNode *p = *head;
    while (p->next != NULL) p = p->next;
    p->next = n;
    n->prev = p;
}

// Forward traversal
void traverseForward(struct DNode *head) {
    while (head != NULL) { printf("%d ", head->data); head = head->next; }
}

// Backward traversal (start from last node, using prev pointers)
void traverseBackward(struct DNode *last) {
    while (last != NULL) { printf("%d ", last->data); last = last->prev; }
}
```

Advantages & Disadvantages of Linked List

Advantages	Disadvantages
Dynamic size — grows/shrinks at run time	No random/direct access (sequential only)
Efficient insertion/deletion (no shifting)	Extra memory needed for pointer field
No memory wastage (allocated as needed)	Traversal is slower than array (pointer chasing)

Advantages	Disadvantages
Easy to implement stacks, queues, etc.	Reverse traversal not possible in SLL

Circular Linked List — Insertion Code

Program / Pseudocode

```
void insertCLL(struct Node **head, int val) {
    struct Node *n = malloc(sizeof(struct Node));
    n->data = val;
    if (*head == NULL) { n->next = n; *head = n; return; }
    struct Node *p = *head;
    while (p->next != *head) p = p->next;    // reach last node
    p->next = n;
    n->next = *head;
}

// Traversal - must use do-while since there is no NULL
void traverseCLL(struct Node *head) {
    if (head == NULL) return;
    struct Node *p = head;
    do {
        printf("%d -> ", p->data);
        p = p->next;
    } while (p != head);
}
```

Sparse Matrix — Array (Triplet) vs Linked List Representation

Basis	Array (Triplet) Representation	Linked List Representation
Structure	3xN array: row, col, value per column	Nodes with row, col, value, next
Size	Fixed (must know non-zero count in advance)	Dynamic — grows as needed
Insertion of new term	Costly (array shifting)	Easy (pointer adjustment)

Polynomial Multiplication (Brief)

To multiply two polynomials represented as linked lists, every term of the first list is multiplied with every term of the second list (coefficients multiply, exponents add), and like terms (same exponent) in the resulting list are combined by adding their coefficients.

Example — Polynomial Multiplication

$$P1 = 2x + 3 \quad P2 = x + 5$$

$$(2x)(x) + (2x)(5) + (3)(x) + (3)(5) = 2x^2 + 10x + 3x + 15 \rightarrow 2x^2 + 13x + 15$$

★ **Exam Tip:** Whenever asked to 'write a program', clearly separate the 3 cases for insertion — at beginning, at end, at a given position — examiners often ask for one specific case, and mixing them up costs marks.

Unit II — Important Exam Questions

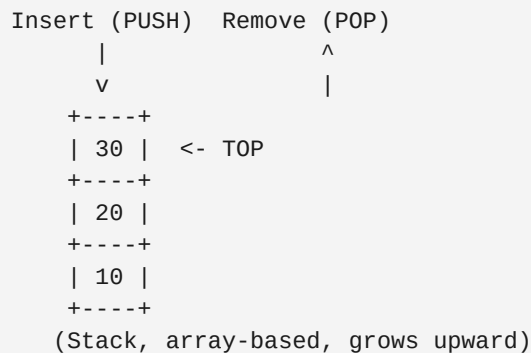
- 1 Define linked list. How is it different from an array?
- 2 Write a C program to insert a node at the beginning/end of a singly linked list.
- 3 Write an algorithm to delete a node from a given position in a linked list.
- 4 What is a doubly linked list? Explain insertion and deletion with a diagram.
- 5 Differentiate between singly, doubly, circular and doubly-circular linked lists.
- 6 Explain sparse matrix representation using a linked list with an example.
- 7 Represent the polynomial $6x^4 + 3x^2 + 5x + 9$ using a linked list and explain polynomial addition.

UNIT III Stacks, Queues & Recursion

3.1 Stack

Definition: A **Stack** is a linear data structure that follows the **LIFO** (Last In First Out) principle — the element inserted last is removed first. Insertion and deletion happen only from one end, called the **TOP**.

Fig 3.1 — Stack (LIFO)



Array Representation of Stack

Program / Pseudocode

```

#define MAX 100
int stack[MAX], top = -1;

void push(int item) {
    if (top == MAX - 1) { printf("Stack Overflow"); return; }
    stack[++top] = item;
}

int pop() {
    if (top == -1) { printf("Stack Underflow"); return -1; }
    return stack[top--];
}

int status() {           // STATUS / PEEK
    if (top == -1) return -1;
    return stack[top];
}
  
```

Linked List Representation of Stack

Program / Pseudocode

```

struct Node { int data; struct Node *next; };
struct Node *top = NULL;

void push(int val) {
    struct Node *n = malloc(sizeof(struct Node));
    n->data = val; n->next = top; top = n;
}

int pop() {
    if (top == NULL) { printf("Underflow"); return -1; }
    int val = top->data;
    struct Node *temp = top;
    top = top->next;
    free(temp);
    return val;
}

```

Basis	Array Stack	Linked List Stack
Size	Fixed (overflow possible)	Dynamic (grows as needed)
Memory	No extra pointer overhead	Extra memory for 'next' pointer
Implementation	Simple	Slightly complex (uses malloc/free)

★ **Exam Tip:** Always mention both PUSH/POP conditions: **Overflow** (stack full) and **Underflow** (stack empty) — examiners specifically check for these boundary conditions.

3.2 Applications: Infix, Postfix & Evaluation

Infix, Prefix, Postfix Notations

Notation	Form	Example (A+B)
Infix	Operator between operands	A + B
Prefix (Polish)	Operator before operands	+ A B
Postfix (Reverse Polish)	Operator after operands	A B +

Infix to Postfix Conversion — Algorithm

- 1 Scan the infix expression from left to right.
- 2 If the scanned symbol is an **operand**, add it directly to the output.
- 3 If the symbol is '(', push it onto the stack.
- 4 If the symbol is ')', pop and output symbols until '(' is popped (discard both parentheses).
- 5 If the symbol is an **operator**, pop and output operators from the stack that have greater than or equal precedence, then push the current operator.
- 6 After the scan, pop and output all remaining operators from the stack.

Example — Infix to Postfix

Convert: $A + B * C - D$

Scan A → output: A | Scan + → push | Scan B → output: A B | Scan * (higher prec. than +) → push

Scan C → output: A B C | Scan - → pop *, pop + (both ≥ prec.) → output: A B C * + , push -

Scan D → output: A B C * + D | End: pop - → Result = $A B C * + D -$

Evaluation of Postfix Expression — Algorithm

- 1 Scan the postfix expression from left to right.
- 2 If the symbol is an **operand**, push it onto the stack.
- 3 If the symbol is an **operator**, pop the top two operands, apply the operator (2nd popped OP 1st popped), and push the result back.
- 4 After the scan, the single value left in the stack is the final result.

Example — Postfix Evaluation

Evaluate: $5\ 3\ 4\ * + 2 -$

Push 5, Push 3, Push 4 → stack: [5,3,4]

'*' → pop 4,3 → $3*4=12$, push 12 → stack: [5,12]

'+' → pop 12,5 → $5+12=17$, push 17 → stack: [17]

Push 2 → stack: [17,2]

'-' → pop 2,17 → $17-2=15$, push 15 → **Result = 15**

3.3 Recursion & Tower of Hanoi

Definition: Recursion is a technique in which a function calls itself, directly or indirectly, to solve a problem by breaking it into smaller sub-problems of the same type. Every recursive function must have a **base case** to stop recursion.

Factorial using Recursion

Program / Pseudocode

```
int factorial(int n) {
    if (n == 0) return 1;           // base case
    return n * factorial(n - 1);    // recursive call
}
```

Fig 3.2 — Recursion Tree for factorial(4)

```
factorial(4)
= 4 * factorial(3)
= 4 * (3 * factorial(2))
= 4 * (3 * (2 * factorial(1)))
= 4 * (3 * (2 * (1 * factorial(0))))
= 4 * (3 * (2 * (1 * 1)))
= 24
```

Fibonacci using Recursion

Program / Pseudocode

```
int fibonacci(int n) {
    if (n == 0) return 0;
    if (n == 1) return 1;
    return fibonacci(n - 1) + fibonacci(n - 2);
}
```

Example — Fibonacci Series

fibonacci(0..6) → 0, 1, 1, 2, 3, 5, 8

Tower of Hanoi

Definition: Tower of Hanoi is a classic recursion problem: move N disks from a **source** peg to a **destination** peg (using an **auxiliary** peg), such that only one disk moves at a time and a larger disk never sits on a smaller one.

Program / Pseudocode

```
void TOH(int n, char src, char aux, char dest) {
    if (n == 0) return;
    TOH(n - 1, src, dest, aux);          // move n-1 disks src -> aux
    printf("Move disk %d from %c to %c\n", n, src, dest);
    TOH(n - 1, aux, src, dest);          // move n-1 disks aux -> dest
}
```

Example — Tower of Hanoi ($n = 2$, $\text{src}=\text{A}$, $\text{dest}=\text{C}$, $\text{aux}=\text{B}$)

Move disk 1 from A to B
 Move disk 2 from A to C
 Move disk 1 from B to C
 Total moves for n disks = $2^n - 1$

★ **Exam Tip:** For Tower of Hanoi, always state the recurrence relation $T(n) = 2T(n-1) + 1$ and the final formula $T(n) = 2^n - 1$. This is a frequently asked derivation.

3.4 Queue

Definition: A **Queue** is a linear data structure that follows the **FIFO** (First In First Out) principle — the element inserted first is removed first. Insertion happens at the **REAR** and deletion happens at the **FRONT**.

Fig 3.3 — Queue (FIFO)

Delete (Dequeue)	Insert (Enqueue)
<----	---->
+---+---+---+---+	
10 20 30 40	
+---+---+---+---+	
FRONT	REAR

Array Representation of Queue

Program / Pseudocode

```
#define MAX 100
int queue[MAX], front = -1, rear = -1;

void enqueue(int item) {
    if (rear == MAX - 1) { printf("Queue Overflow"); return; }
    if (front == -1) front = 0;
    queue[++rear] = item;
}

int dequeue() {
    if (front == -1 || front > rear) { printf("Queue Underflow"); return -1; }
    int item = queue[front++];
    return item;
}
```

Linked List Representation of Queue**Program / Pseudocode**

```
struct Node { int data; struct Node *next; };
struct Node *front = NULL, *rear = NULL;

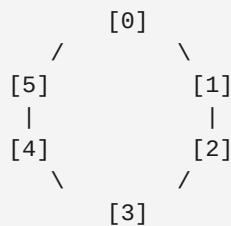
void enqueue(int val) {
    struct Node *n = malloc(sizeof(struct Node));
    n->data = val; n->next = NULL;
    if (rear == NULL) { front = rear = n; return; }
    rear->next = n;
    rear = n;
}

int dequeue() {
    if (front == NULL) { printf("Underflow"); return -1; }
    int val = front->data;
    struct Node *temp = front;
    front = front->next;
    if (front == NULL) rear = NULL;
    free(temp);
    return val;
}
```

3.5 Circular Queue, Deque & Priority Queue**Circular Queue**

Definition: A **Circular Queue** connects the rear back to the front, forming a circle, so that the empty space created after dequeue operations can be reused — solving the problem of unnecessary Overflow in a simple array queue.

Fig 3.4 — Circular Queue



(Rear wraps back to Front position 0 when it reaches the end)

```
rear = (rear + 1) % MAX
front = (front + 1) % MAX
```

Deque (Double-Ended Queue)

Definition: A **Deque** allows insertion and deletion from **both ends** — front and rear. Two types: **Input-restricted deque** (insertion only at one end, deletion at both) and **Output-restricted deque** (deletion only at one end, insertion at both).

Priority Queue

Definition: A **Priority Queue** is a queue in which each element has a **priority**. Elements with higher priority are served before elements with lower priority, regardless of their order of arrival. Often implemented using a Heap.

Queue Type	Insertion	Deletion	Key Feature
Simple Queue	Rear only	Front only	Basic FIFO
Circular Queue	Rear (wraps around)	Front (wraps around)	Reuses empty slots
Deque	Both ends	Both ends	Flexible both-end access
Priority Queue	Based on priority	Highest priority first	Not strict FIFO

★ **Exam Tip:** Stack vs Queue is a guaranteed 2–3 mark question: Stack = LIFO (one end), Queue = FIFO (two ends: front & rear). Applications: Stack → recursion, expression evaluation, undo operations, backtracking. Queue → CPU scheduling, printer spooling, BFS traversal.

3.6 Additional Concepts — Full Traces & More Applications

Applications of Stack (Complete List)

- Expression evaluation and conversion (infix ↔ postfix ↔ prefix).
- Balancing/matching of parentheses and symbols in an expression or code editor.
- Function call management — every function call is pushed on the **call stack**; recursion is implemented internally using a stack.
- Undo/Redo operations in text editors and software.
- Backtracking algorithms — maze solving, N-Queens problem.
- Syntax parsing in compilers.

Checking Balanced Parentheses using Stack — Algorithm

Program / Pseudocode

```
int isBalanced(char *expr) {
    char stack[100]; int top = -1;
    for (int i = 0; expr[i] != '\0'; i++) {
        char c = expr[i];
        if (c == '(' || c == '{' || c == '[')
            stack[++top] = c;
        else if (c == ')' || c == '}' || c == ']') {
            if (top == -1) return 0; // unbalanced
            char open = stack[top--];
            if ((c == ')' && open != '(') || (c == '}' && open != '{') ||
                (c == ']' && open != '['))
                return 0; // mismatched
        }
    }
    return (top == -1); // balanced only if stack is empty at the end
}
```

Example — Balanced Check

Expression: { [()] } → every open symbol finds its matching close symbol in correct order and stack becomes empty → **Balanced**. Expression { [(]) } → mismatch when ']' tries to close '(' → **Not Balanced**.

Full Step-by-Step Trace — Infix to Postfix

Convert: (A + B) * (C - D)

Symbol	Action	Stack (top→right)	Output so far
(	push	(	
A	output	(	A
+	push (after '(' — nothing to pop)	(+	A
B	output	(+	A B
)	pop till '('	(empty)	A B +
*	push	*	A B +
(	push	* (	A B +
C	output	* (	A B + C
-	push	* (-	A B + C
D	output	* (-	A B + C D
)	pop till '('	*	A B + C D -
end	pop remaining	(empty)	A B + C D - *

Final Postfix: A B + C D - *

Full Step-by-Step Trace — Postfix Evaluation

Evaluate: **6 3 2 * 7 - +**

Symbol	Action	Stack after action
6	push	6
3	push	6, 3
2	push	6, 3, 2
*	pop 2,3 → 3*2=6, push	6, 6
7	push	6, 6, 7
-	pop 7,6 → 6-7=-1, push	6, -1
+	pop -1,6 → 6+(-1)=5, push	5

Final Result = 5

Types of Recursion

Type	Description
Direct Recursion	A function calls itself directly (e.g. factorial(n) calling factorial(n-1))
Indirect Recursion	Function A calls function B, and B calls A back
Tail Recursion	The recursive call is the very last statement in the function — can be optimised into a loop
Non-Tail Recursion	Some computation remains after the recursive call returns (e.g. n * factorial(n-1))

Circular Queue — Full Enqueue/Dequeue Code with Modulo

Program / Pseudocode

```
#define MAX 5
int cq[MAX], front = -1, rear = -1;

void enqueue(int val) {
    if ((rear + 1) % MAX == front) { printf("Overflow"); return; }
    if (front == -1) front = 0;
    rear = (rear + 1) % MAX;
    cq[rear] = val;
}

int dequeue() {
    if (front == -1) { printf("Underflow"); return -1; }
    int val = cq[front];
    if (front == rear) front = rear = -1; // queue becomes empty
    else front = (front + 1) % MAX;
    return val;
}
```

Applications of Queue

- CPU / process scheduling (Round-Robin scheduling in OS).
- Handling of requests on a single shared resource, e.g. printer spooling.
- Breadth-First Search (BFS) traversal of trees and graphs.
- Maintaining playlists, call-centre systems (first come, first served).
- Buffering of data streams (IO buffers, keystroke buffers).

★ **Exam Tip:** Balanced-parentheses checking and the full infix-postfix / postfix-evaluation trace TABLES (as shown above) are extremely high-yield — practice writing the table format itself since many university answer keys award marks strictly for the tabular trace format.

Unit III — Important Exam Questions

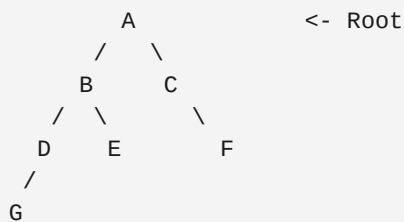
- 1 Define stack. Explain PUSH and POP operations with array representation.
- 2 Convert the infix expression $(A + B) * (C - D) / E$ into postfix, showing the stack status at each step.
- 3 Evaluate the postfix expression $6\ 2\ 3\ +\ -\ 4\ *$ using a stack.
- 4 What is recursion? Write a recursive program for factorial and explain the recursion tree.
- 5 Explain the Tower of Hanoi problem with recursive algorithm and derive its time complexity.
- 6 Define queue. Differentiate between simple queue and circular queue.
- 7 What is a deque? Explain its types with diagrams.
- 8 Explain priority queue with a suitable example.

UNIT IV Trees & Graphs

4.1 Tree Terminology & Binary Tree

Definition: A **Tree** is a non-linear, hierarchical data structure consisting of nodes connected by edges, with one node designated as the **root**. Each node may have zero or more child nodes, and there are no cycles.

Fig 4.1 — A General Tree



Level 0 : A
 Level 1 : B, C
 Level 2 : D, E, F
 Level 3 : G

Basic Terminology

Term	Meaning
Root	The topmost node of the tree (no parent)
Node/Vertex	An element of the tree containing data
Edge	Link connecting a parent node to a child node
Parent	A node that has one or more child nodes
Child	A node directly connected to a parent node below it
Leaf / Terminal Node	A node with no children
Siblings	Nodes having the same parent
Degree of a node	Number of children of that node
Height of tree	Number of edges on the longest path from root to a leaf
Depth/Level of a node	Number of edges from the root to that node
Subtree	A tree formed by a node and all its descendants

Binary Tree — Properties

Definition: A **Binary Tree** is a tree in which every node has **at most two children**, referred to as the **left child** and **right child**.

- Maximum number of nodes at level 'l' = 2^l .
- Maximum number of nodes in a binary tree of height 'h' = $2^{h+1} - 1$.
- Minimum number of nodes possible in a binary tree of height 'h' = h + 1.
- A binary tree with 'n' nodes has exactly (n - 1) edges.
- **Full/Strictly Binary Tree:** every node has 0 or 2 children.
- **Complete Binary Tree:** all levels are completely filled except possibly the last, which is filled from left to right.

Representation of Binary Tree

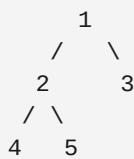
- **Array (Sequential/Linear) Representation:** Store nodes in an array using the rule — for a node at index i, left child is at (2i+1) and right child at (2i+2) (0-indexed), parent at ((i-1)/2).
- **Linked Representation:** Each node is a structure with data, a left-child pointer, and a right-child pointer.

Program / Pseudocode

```
struct TreeNode {
    int data;
    struct TreeNode *left, *right;
};
```

4.2 Tree Traversals

Fig 4.2 — Sample Binary Tree for Traversal Examples



(a) Inorder Traversal (Left - Root - Right)

Program / Pseudocode

```
void inorder(struct TreeNode *root) {
    if (root == NULL) return;
    inorder(root->left);
    printf("%d ", root->data);
    inorder(root->right);
}
```

Example — Inorder of Fig 4.2

Traversal order: 4, 2, 5, 1, 3

(b) Preorder Traversal (Root - Left - Right)

Program / Pseudocode

```
void preorder(struct TreeNode *root) {
    if (root == NULL) return;
    printf("%d ", root->data);
    preorder(root->left);
    preorder(root->right);
}
```

Example — Preorder of Fig 4.2

Traversal order: 1, 2, 4, 5, 3

(c) Postorder Traversal (Left - Right - Root)

Program / Pseudocode

```
void postorder(struct TreeNode *root) {
    if (root == NULL) return;
    postorder(root->left);
    postorder(root->right);
    printf("%d ", root->data);
}
```

Example — Postorder of Fig 4.2

Traversal order: 4, 5, 2, 3, 1

★ **Exam Tip:** Trick to remember: **In**order → root is **In** the middle. **Pre**order → root comes first (**P**refix style). **Post**order → root comes last (**P**ostfix style). Drawing the tree and writing traversal step-by-step gets full marks even if you forget the code.

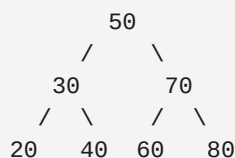
4.3 BST, AVL Tree & M-Way Search Tree

Binary Search Tree (BST)

Definition: A **Binary Search Tree** is a binary tree with the property: for every node, all values in its **left subtree** are **smaller**, and all values in its **right subtree** are **greater** than the node's value. This allows fast searching, similar to binary search.

Fig 4.3 — Binary Search Tree

Insert: 50, 30, 70, 20, 40, 60, 80



Inorder traversal of a BST always gives SORTED order: 20 30 40 50 60 70 80

- Search, Insert, Delete: $O(\log n)$ average case, $O(n)$ worst case (skewed tree).

AVL Tree

Definition: An **AVL Tree** (Adelson-Velsky & Landis) is a **self-balancing** Binary Search Tree in which the difference of heights (Balance Factor) of the left and right subtrees of any node is at most 1.

AVL — Balance Factor & Rotations

Balance Factor (BF) = Height(Left Subtree) – Height(Right Subtree)

Valid BF values for AVL tree: -1, 0, +1

If BF becomes -2 or +2 after insertion/deletion, rotations are performed:

1. LL Rotation (Single Right Rotation)
2. RR Rotation (Single Left Rotation)
3. LR Rotation (Left-Right, Double Rotation)
4. RL Rotation (Right-Left, Double Rotation)

AVL trees guarantee $O(\log n)$ time for search, insert and delete in the **worst case** as well, unlike a plain BST — this is its main advantage.

M-Way Search Tree

Definition: An **M-Way Search Tree** is a generalised search tree in which each node can have **up to M children** and up to (M–1) keys, with keys within a node kept in sorted order. It is the foundation for **B-Trees**, widely used in databases and file systems for efficient disk-based searching.

Tree Type	Balancing	Search Time (Worst)	Typical Use
Binary Search Tree	Not self-balancing	$O(n)$	Simple ordered storage
AVL Tree	Self-balancing	$O(\log n)$	Frequently searched data
M-Way Search Tree	Multi-key nodes	$O(\log n)$	Databases, file systems (B-Trees)

Applications of Trees

- Representing hierarchical data (file systems, organisation charts).
- Binary Search Trees — fast searching, insertion, deletion of sorted data.
- Expression Trees — representing and evaluating arithmetic expressions.
- Huffman coding trees — data compression.
- AVL / B-Trees — databases and indexing for quick disk access.
- Decision trees — used in AI and machine learning.

4.4 Graph — Terminology & Representation

Definition: A **Graph** $G = (V, E)$ is a non-linear data structure consisting of a set of **vertices (V)** and a set of **edges (E)** connecting pairs of vertices. Unlike a tree, a graph can have cycles and a vertex can connect to any number of other vertices.

Fig 4.4 — Sample Graph


 $V = \{1, 2, 3, 4\}$
 $E = \{(1,2), (1,3), (1,4), (2,3), (3,4)\}$

Graph Terminology

Term	Meaning
Vertex	A node of the graph
Edge	A connection between two vertices
Directed Graph	Edges have a direction (one-way)
Undirected Graph	Edges have no direction (two-way)
Weighted Graph	Each edge has an associated cost/weight
Degree of vertex	Number of edges connected to that vertex
Path	A sequence of vertices connected by edges, no repeats
Cycle	A path that starts and ends at the same vertex
Connected Graph	A path exists between every pair of vertices

Representations of a Graph

(a) Adjacency Matrix: A $V \times V$ matrix where cell $(i, j) = 1$ (or weight) if an edge exists between vertex i and j , else 0.

Adjacency Matrix

For the graph in Fig 4.4:

```

      1  2  3  4
1[  0  1  1  1 ]
2[  1  0  1  0 ]
3[  1  1  0  1 ]
4[  1  0  1  0 ]

```

(b) Adjacency List: Each vertex has a linked list of its adjacent (directly connected) vertices — saves memory for sparse graphs.

Adjacency List

```

1 -> 2 -> 3 -> 4 -> NULL
2 -> 1 -> 3 -> NULL
3 -> 1 -> 2 -> 4 -> NULL
4 -> 1 -> 3 -> NULL

```

(c) Adjacency Set: Similar to adjacency list, but uses a set data structure for each vertex's neighbours — allows $O(1)$ average edge-existence checks.

Representation	Space	Best suited for
Adjacency Matrix	$O(V^2)$	Dense graphs, quick edge lookup
Adjacency List	$O(V + E)$	Sparse graphs, memory-efficient
Adjacency Set	$O(V + E)$	Fast edge existence check

4.5 Graph Traversal: BFS & DFS

Breadth-First Search (BFS)

Definition: BFS explores a graph **level by level** — visiting all neighbours of a vertex before moving to the next level. It uses a **Queue** data structure.

Program / Pseudocode

```
void BFS(int start, int V, int adj[V][V]) {
    int visited[V] = {0};
    int queue[V], front = 0, rear = 0;
    visited[start] = 1;
    queue[rear++] = start;
    while (front < rear) {
        int u = queue[front++];
        printf("%d ", u);
        for (int v = 0; v < V; v++)
            if (adj[u][v] && !visited[v]) {
                visited[v] = 1;
                queue[rear++] = v;
            }
    }
}
```

Example — BFS on Fig 4.4 (start = 1)

Order visited: **1, 2, 3, 4**

Depth-First Search (DFS)

Definition: DFS explores a graph by going as **deep as possible** along one path before backtracking. It uses a **Stack** (or recursion).

Program / Pseudocode

```
void DFS(int u, int V, int adj[V][V], int visited[]) {
    visited[u] = 1;
    printf("%d ", u);
    for (int v = 0; v < V; v++)
        if (adj[u][v] && !visited[v])
            DFS(v, V, adj, visited);
}
```

Example — DFS on Fig 4.4 (start = 1)

Order visited: **1, 2, 3, 4** (depends on adjacency order; here $1 \rightarrow 2 \rightarrow 3 \rightarrow 4$ since 3 connects back to 1 & 2 already visited)

Basis	BFS	DFS
Data Structure Used	Queue	Stack / Recursion
Approach	Level-wise (breadth)	Path-wise (depth)
Time Complexity	$O(V + E)$	$O(V + E)$
Space Complexity	$O(V)$	$O(V)$
Shortest Path (unweighted)	Yes	No
Applications	Shortest path, GPS navigation, social networks	Cycle detection, topological sort, maze solving

Applications of Graphs

- Representing networks: computer networks, social networks, road/flight maps.
- Shortest path algorithms (Dijkstra, Bellman-Ford) built on graph traversal.
- Web page ranking (Google PageRank) uses directed graphs.
- Detecting cycles/deadlocks in operating systems (resource allocation graphs).
- Scheduling problems using topological sorting.

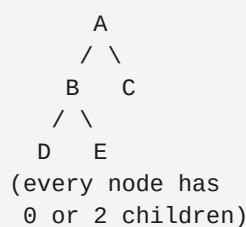
★ **Exam Tip:** BFS & DFS numericals (given a graph, list traversal order starting from a given vertex) are almost guaranteed. Practice drawing the adjacency list first, then simulate the queue (BFS) / stack (DFS) step by step on paper.

4.6 Additional Concepts — Types, Codes & Full Traces

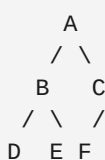
Types of Binary Trees

Fig 4.5 — Types of Binary Trees

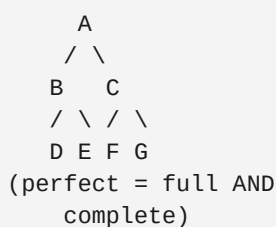
Full / Strictly Binary Tree (0 or 2 children)



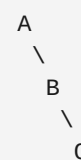
Complete Binary Tree (filled left to right)



Perfect Binary Tree (all leaves same level)



Skewed Tree (only left/right)



Type	Rule
Full / Strictly Binary Tree	Every node has exactly 0 or 2 children
Complete Binary Tree	All levels full except possibly the last, filled left to right
Perfect Binary Tree	All internal nodes have 2 children AND all leaves are at the same level

Type	Rule
Skewed Tree	Every node has only a left child (left-skewed) or only a right child (right-skewed)
Balanced Tree	Height difference between left and right subtree is bounded (e.g. AVL)

Threaded Binary Tree (Brief)

Definition: A **Threaded Binary Tree** makes use of the NULL left/right pointers in a binary tree by replacing them with '**threads**' — pointers to the inorder predecessor/successor. This allows traversal without recursion or an explicit stack, and without extra memory.

BST — Search, Insert & Delete (Full Code)

Program / Pseudocode

```

struct TreeNode* insertBST(struct TreeNode *root, int key) {
    if (root == NULL) {
        struct TreeNode *n = malloc(sizeof(struct TreeNode));
        n->data = key; n->left = n->right = NULL;
        return n;
    }
    if (key < root->data) root->left = insertBST(root->left, key);
    else if (key > root->data) root->right = insertBST(root->right, key);
    return root;
}

struct TreeNode* search(struct TreeNode *root, int key) {
    if (root == NULL || root->data == key) return root;
    if (key < root->data) return search(root->left, key);
    return search(root->right, key);
}

// Delete: 3 cases -> leaf node, one child, two children
struct TreeNode* deleteBST(struct TreeNode *root, int key) {
    if (root == NULL) return root;
    if (key < root->data) root->left = deleteBST(root->left, key);
    else if (key > root->data) root->right = deleteBST(root->right, key);
    else {
        if (root->left == NULL) return root->right;        // 0 or 1 child
        if (root->right == NULL) return root->left;
        struct TreeNode *succ = root->right;              // 2 children:
        while (succ->left != NULL) succ = succ->left;      // find inorder successor
        root->data = succ->data;
        root->right = deleteBST(root->right, succ->data);
    }
    return root;
}

```

BST Deletion — Three Cases (Diagram)

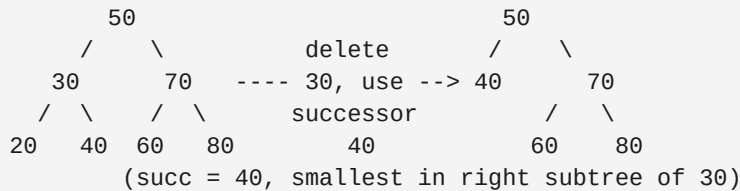
Fig 4.6 — BST Deletion Cases

Case 1: Node is a LEAF
Simply remove it.

Case 2: Node has ONE child
Replace node with its child.

Case 3: Node has TWO children
Replace node's value with its INORDER SUCCESSOR
(smallest value in right subtree), then delete that successor.

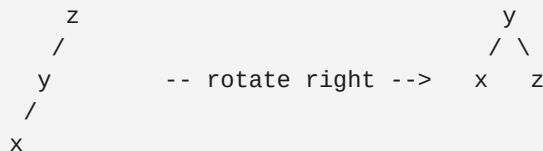
Example: Delete 30 from:



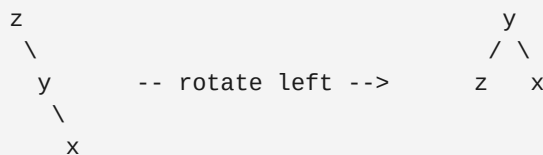
AVL Tree — Rotation Diagrams

Fig 4.7 — Four AVL Rotation Types

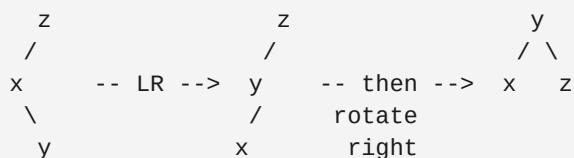
LL Rotation (Right Rotation) - inserted in Left-Left:



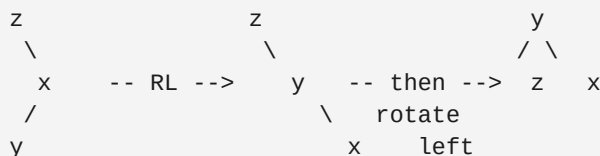
RR Rotation (Left Rotation) - inserted in Right-Right:



LR Rotation (Left-Right) - inserted in Left-Right:



RL Rotation (Right-Left) - inserted in Right-Left:



Example — AVL Rotation Example

Insert 30, 20, 10 into an empty AVL tree.

Insert 30 → [30]. Insert 20 → [30(20,-)]. Insert 10 → node 30 gets BF = +2 (Left-Left case) → apply **LL (single right) rotation** → final balanced tree: **20** as root with **10** (left) and **30** (right).

Types of Graphs

Type	Description
Simple Graph	No self-loops and no multiple edges between same pair of vertices
Multigraph	Multiple edges allowed between the same pair of vertices
Complete Graph	Every pair of distinct vertices is connected (n vertices $\rightarrow n(n-1)/2$ edges)
Directed Graph (Digraph)	Every edge has a direction
Weighted Graph	Every edge carries a numeric weight/cost
Cyclic / Acyclic Graph	Contains at least one cycle / contains no cycle

Full Step-by-Step Trace — BFS

Graph from Fig 4.4, adjacency list: $1 \rightarrow [2,3,4]$, $2 \rightarrow [1,3]$, $3 \rightarrow [1,2,4]$, $4 \rightarrow [1,3]$. Start vertex = 1.

Step	Dequeue (visit)	Queue after processing	Visited so far
1	— (initial)	[1]	{ }
2	1	[2, 3, 4]	{1}
3	2	[3, 4] (3 already queued, skip)	{1,2}
4	3	[4] (4 already queued, skip)	{1,2,3}
5	4	[] (empty)	{1,2,3,4}

BFS order = 1, 2, 3, 4

Full Step-by-Step Trace — DFS

Step	Action	Stack / Call state	Visited so far
1	Visit 1	call DFS(1)	{1}
2	Visit 2 (1st unvisited neighbour of 1)	call DFS(2)	{1,2}
3	Visit 3 (unvisited neighbour of 2)	call DFS(3)	{1,2,3}
4	Visit 4 (unvisited neighbour of 3)	call DFS(4)	{1,2,3,4}
5	No unvisited neighbours — backtrack fully	returns all the way	{1,2,3,4}

DFS order = 1, 2, 3, 4

★ **Exam Tip:** For BFS/DFS trace questions, ALWAYS show the queue/stack contents at every step in a table like above — examiners give step-wise marks even if your final order has a minor error.

Unit IV — Important Exam Questions

- 1 Define tree. Explain the basic terminology used in trees with a diagram.
- 2 What is a binary tree? State and prove the maximum number of nodes at level l and in a tree of height h .
- 3 Write recursive algorithms for inorder, preorder and postorder traversal. Apply them on a given binary tree.

- 4 What is a Binary Search Tree? Construct a BST by inserting: 45, 15, 79, 90, 10, 55, 12, 20, 50.
- 5 Explain AVL tree and balance factor. Describe the four types of rotations.
- 6 What is an M-way search tree? How is it related to B-Trees?
- 7 Define graph. Explain adjacency matrix and adjacency list representation with an example.
- 8 Differentiate between BFS and DFS traversal with an example graph.
- 9 Write the BFS/DFS algorithm and trace it on a given graph starting from a specified vertex.

QUICK REVISION

Complexity Cheat-Sheet & Final Recap

Time & Space Complexity — Master Table

Data Structure / Algorithm	Access	Search	Insert	Delete	Space
Array	$O(1)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$
Singly Linked List	$O(n)$	$O(n)$	$O(1)^*$	$O(1)^*$	$O(n)$
Doubly Linked List	$O(n)$	$O(n)$	$O(1)^*$	$O(1)^*$	$O(n)$
Stack	$O(n)$	$O(n)$	$O(1)$	$O(1)$	$O(n)$
Queue	$O(n)$	$O(n)$	$O(1)$	$O(1)$	$O(n)$
Binary Search Tree (avg)	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(n)$
Binary Search Tree (worst)	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$
AVL Tree	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(n)$
Graph (Adjacency Matrix)	—	$O(V^2)$	$O(1)$	$O(1)$	$O(V^2)$
Graph (Adjacency List)	—	$O(V+E)$	$O(1)$	$O(V)$	$O(V+E)$

* $O(1)$ once the position/pointer is known; $O(n)$ if the position must be searched first.

Sorting Algorithms Recap

Algorithm	Best	Average	Worst	Space
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$

Searching Algorithms Recap

Algorithm	Best	Worst	Precondition
Linear Search	$O(1)$	$O(n)$	None
Binary Search	$O(1)$	$O(\log n)$	Array must be sorted

One-Line Definitions to Remember

- **Array:** Contiguous, homogeneous, fixed-size, direct-access collection.
- **Linked List:** Dynamic, non-contiguous, sequential-access nodes linked by pointers.
- **Stack:** LIFO — insertion/deletion from one end (TOP).
- **Queue:** FIFO — insertion at REAR, deletion at FRONT.
- **Tree:** Hierarchical, non-linear, no cycles, one root.
- **Graph:** Non-linear, vertices + edges, cycles allowed.

- **BST:** Left < Root < Right for every node.
- **AVL Tree:** Self-balancing BST, balance factor $\in \{-1, 0, 1\}$.
- **BFS:** Level-order using Queue. **DFS:** Depth-order using Stack/Recursion.

Last-Minute Exam Strategy

- 1 For every definition-based question, first write the exact definition, then one line of explanation, then a diagram if possible — diagrams fetch extra marks.
- 2 For 'write a program/algorithm' questions, always mention the overflow/underflow or base-case conditions — examiners specifically look for these.
- 3 For numericals (address calculation, dry runs of sorting, BFS/DFS, infix-postfix), show every step — partial marks are given for correct intermediate steps.
- 4 Practice drawing: Array insertion/deletion, Linked List node structures, Stack/Queue, Binary Tree traversal order, Graph adjacency list/matrix — these diagrams repeat across most university papers.
- 5 Revise the Complexity Cheat-Sheet on this page one final time before entering the exam hall.

End of Notes — Compiled by Manthan Sahu, Rajendra University, Balangir. All the best for your examination!